

Check whether a PDF file is valid with Python

Asked 15 years, 9 months ago Modified 9 months ago Viewed 53k times

▲ **I get a file via a HTTP upload and need to make sure its a PDF file.** The programming language is Python, but this should not matter.

27

I thought of the following solutions:

- ▼
- 1. Check if the first bytes of the string are `%PDF` . *This is not a good check but prevents the user from uploading other files accidentally.*
 - 2. Use `libmagic` (the `file` command in `bash` uses it). *This does exactly the same check as in (1)*
 - 3. Use a library to try to read the page count out of the file. *If the lib is able to read a page count it should be a valid PDF file. Problem: I don't know a Python library that can do this*

Are there solutions using a library or another trick?

`python` `file` `pdf`

[Share](#) [Improve this question](#) [Follow](#)

edited Jan 20, 2023 at 19:39

asked Feb 17, 2009 at 22:53



WoJ

29.7k

58

210

391



theomega

32k

21

92

128

8 Answers

Sorted by: Highest score (default) ▼

▲ The current solution (as of 2023) is to use [pypdf](#) and catch exceptions (and possibly analyze [reader.metadata](#))

31

▼

```
from pypdf import PdfReader
from pypdf.errors import PdfReadError

with open("testfile.txt", "w") as f:
    f.write("hello world!")

try:
    PdfReader("testfile.txt")
except PdfReadError:
    print("invalid PDF file")
```

```
else:  
    pass
```

Share Improve this answer Follow

edited Jan 20, 2023 at 17:59

answered Sep 18, 2015 at 14:36



Martin Thoma

135k 172 670 1k



Woj

29.7k 58 210 391

- 1 This gives me false positives for zip files. Error message is: `incorrect startxref pointer(1)`
– JulianWgs Mar 25, 2023 at 11:30
- 3 Use PdfReader("testfile.txt", strict=True) and a bare except. – JulianWgs Mar 25, 2023 at 12:05



15



The two most commonly used PDF libraries for Python are:

- [pypdf](#)
- [ReportLab](#)

Both are pure python so should be easy to install as well be cross-platform.

With pypdf it would probably be as simple as doing:

```
from pypdf import PdfReader  
reader = PdfReader("upload.pdf")
```

This should be enough, but `reader` will now have the `metadata` and `pages` attributes if you want to do further checking.

As Carl answered, `pdftotext` is also a good solution, and would probably be faster on very large documents (especially ones with many cross-references). However it might be a little slower on small PDF's due to system overhead of forking a new process, etc.

Share Improve this answer Follow

edited Apr 23, 2023 at 10:32

answered Feb 18, 2009 at 1:10



Martin Thoma

135k 172 670 1k



Van Gale

43.9k 9 74 81



14



In a project if mine I need to check for the mime type of some uploaded file. I simply use the file command like this:

```
from subprocess import Popen, PIPE  
filetype = Popen("/usr/bin/file -b --mime -", shell=True, stdout=PIPE,  
stdin=PIPE).communicate(file.read(1024))[0].strip()
```

You of course might want to move the actual command into some configuration file as also command line options vary among operating systems (e.g. mac).

If you just need to know whether it's a PDF or not and do not need to process it anyway I think the file command is a faster solution than a lib. Doing it by hand is of course also possible but the file command gives you maybe more flexibility if you want to check for different types.

Share Improve this answer Follow

answered Feb 17, 2009 at 23:19



MrTopf

4,843 2 25 19

+1 for simplicity. If you just want to be fairly sure what you've got is at least trying to be a PDF this is a both simple and speedy. – [Roger Heathcote](#) May 1, 2013 at 22:04

This is NOT a solution since it does not work for all pdf files. I have one broken file (unable to read in Adobe Reader, evince, ..), but file -b --mime returns application/pdf; charset=binary. – [rosch](#) Jun 16, 2022 at 14:32



3

If you're on a Linux or OS X box, you could use [Pdftotext](#) (part of Xpdf, found [here](#)). If you pass a non-PDF to pdftotext, it will certainly bark at you, and you can use `commands.getstatusoutput` to get the output and parse it for these warnings.



If you're looking for a platform-independent solution, you might be able to make use of [pypdf](#).



Edit: It's not elegant, but it looks like pypdf's PdfReader will throw an `IOError(22)` if you attempt to load a non-PDF.

Share Improve this answer Follow

edited Apr 23, 2023 at 10:33

answered Feb 17, 2009 at 23:00



Martin Thoma

135k 172 670 1k



Cal Jacobson

2,407 1 26 35



2

Here is a solution using [pdfminersix](#), which can be installed with `pip install pdfminer.six`:



```
from pdfminer.high_level import extract_text

def is_pdf(path_to_file):
    try:
        extract_text(path_to_file)
        return True
    except:
        return False
```



You can also use [filetype](#) (`pip install filetype`):

```
import filetype

def is_pdf(path_to_file):
    return filetype.guess(path_to_file).mime == 'application/pdf'
```

Neither of these solutions is ideal.

1. The problem with the `filetype` solution is that it doesn't tell you if the PDF itself is *readable* or not. It will tell you if the file is a PDF, but it could be a corrupt PDF.
2. The `pdfminer` solution should only return `True` if the PDF is actually readable. But it is a big library and seems like overkill for such a simple function.

I've started another [thread here](#) asking how to check if a file is a valid PDF without using a library (or using a smaller one).

Share Improve this answer Follow

answered Oct 8, 2020 at 22:25

 **Webucator**
2,663 2 28 41

What about this solution using pypdf? gist.github.com/gvangool/129962/... Would it be much less resource intensive than pdfminer.six since it is only creating a reader? – [Ryan Eom](#) Nov 10, 2020 at 2:18



2

I run into the same problem but was not forced to use a programming language to manage this task. I used pypdf but was not efficient for me as it hangs infinitely on some corrupted files.

However, I found this software useful till now.



Good luck with it.



<https://sourceforge.net/projects/corruptedpdffinder/>

Share Improve this answer Follow

edited Apr 23, 2023 at 10:33



Martin Thoma
135k 172 670 1k

answered Jun 16, 2019 at 7:47



Maged Saeed
1,875 2 18 35



0

In case anyone else is having a similar setup as me using **PyMuPDF** (imported as `fitz` package) and **FastAPI**, this works out for me



```
import fitz
import os
from FastAPI import HTTPException
import tempfile

@app.post("/upload")
def upload(file: UploadFile = File(...)):
    """Upload file endpoint."""

    # 1. Check if the file is a PDF
    if file.content_type != "application/pdf":
        raise HTTPException(
            status_code=400,
```



```

        detail="File is not a PDF!",
    )

# 2. Check that the file ending is correct
if not file.filename.endswith(".pdf"):
    raise HTTPException(
        status_code=400,
        detail="Filename does not end as PDF!",
    )

# 3. Check reading it with fitz throws no error
try:
    # Save the uploaded file temporarily
    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        tmp.write(file.file.read())
        # Set the file pointer to the beginning of the file to be able to
read it again later on
        file.file.seek(0)
        tmp_path = tmp.name

    # Check if the file size is greater than 0 (i.e., not empty)
    if os.path.getsize(tmp_path) == 0:
        raise HTTPException(status_code=400, detail="Uploaded file is empty")

    # Open the file to check if it is a valid PDF via the fitz library
    doc = fitz.open(tmp_path)
    doc.close()

    # Clean up the temporary file
    os.unlink(tmp_path)
except HTTPException as e:
    raise HTTPException(
        status_code=400, detail="Uploaded file is not a valid PDF"
    ) from e

```

Share Improve this answer Follow

answered Feb 28 at 17:12



Leon

602

1

6

15



By valid do you mean that it can be displayed by a PDF viewer, or that the text can be extracted? They are two very different things.

-1



If you just want to check that it really is a PDF file that has been uploaded then the pypdf solution, or something similar, will work.



If, however, you want to check that the text can be extracted then you have found a whole world of pain! Using pdftotext would be a simple solution that would work in a majority of cases but it is by no means 100% successful. We have found many examples of PDFs that pdftotext cannot extract from but Java libraries such as iText and PDFBox can.

Share Improve this answer Follow

edited Apr 23, 2023 at 10:35



Martin Thoma

answered Feb 25, 2009 at 0:10



Steve Claridge

